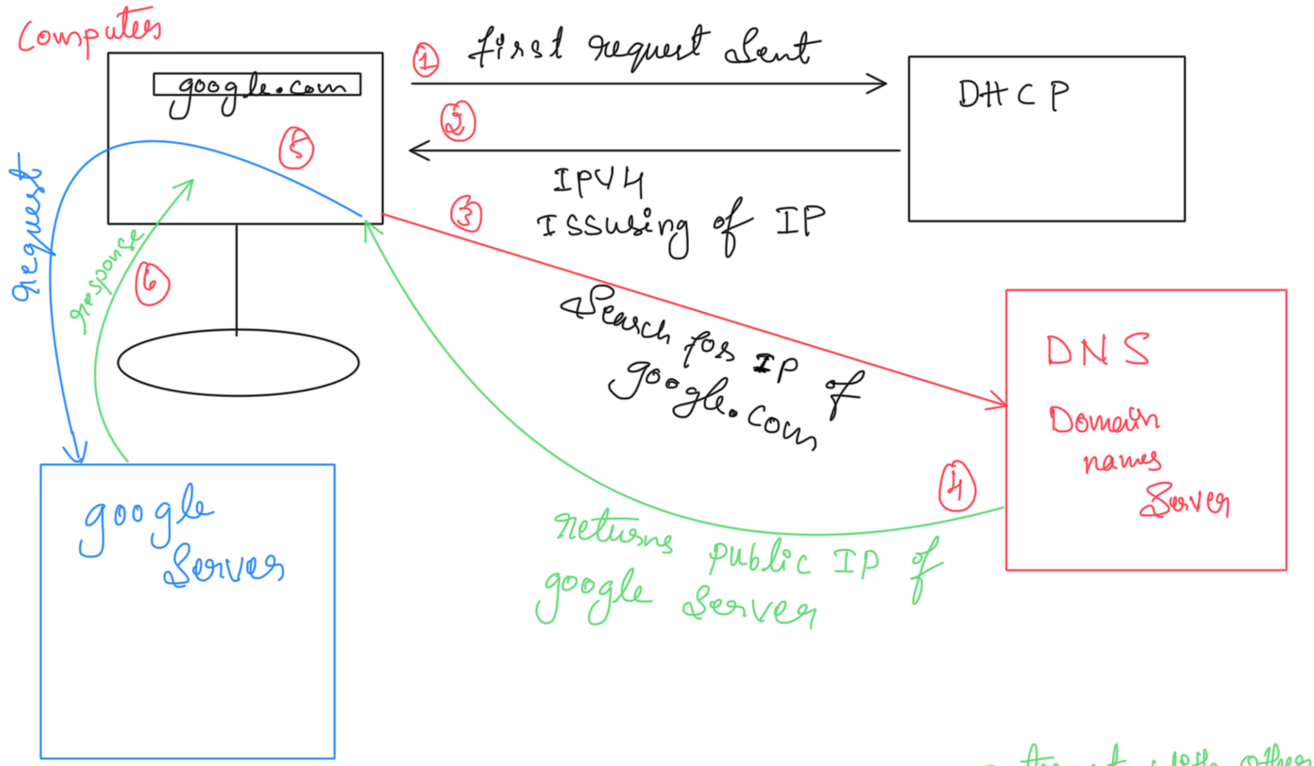
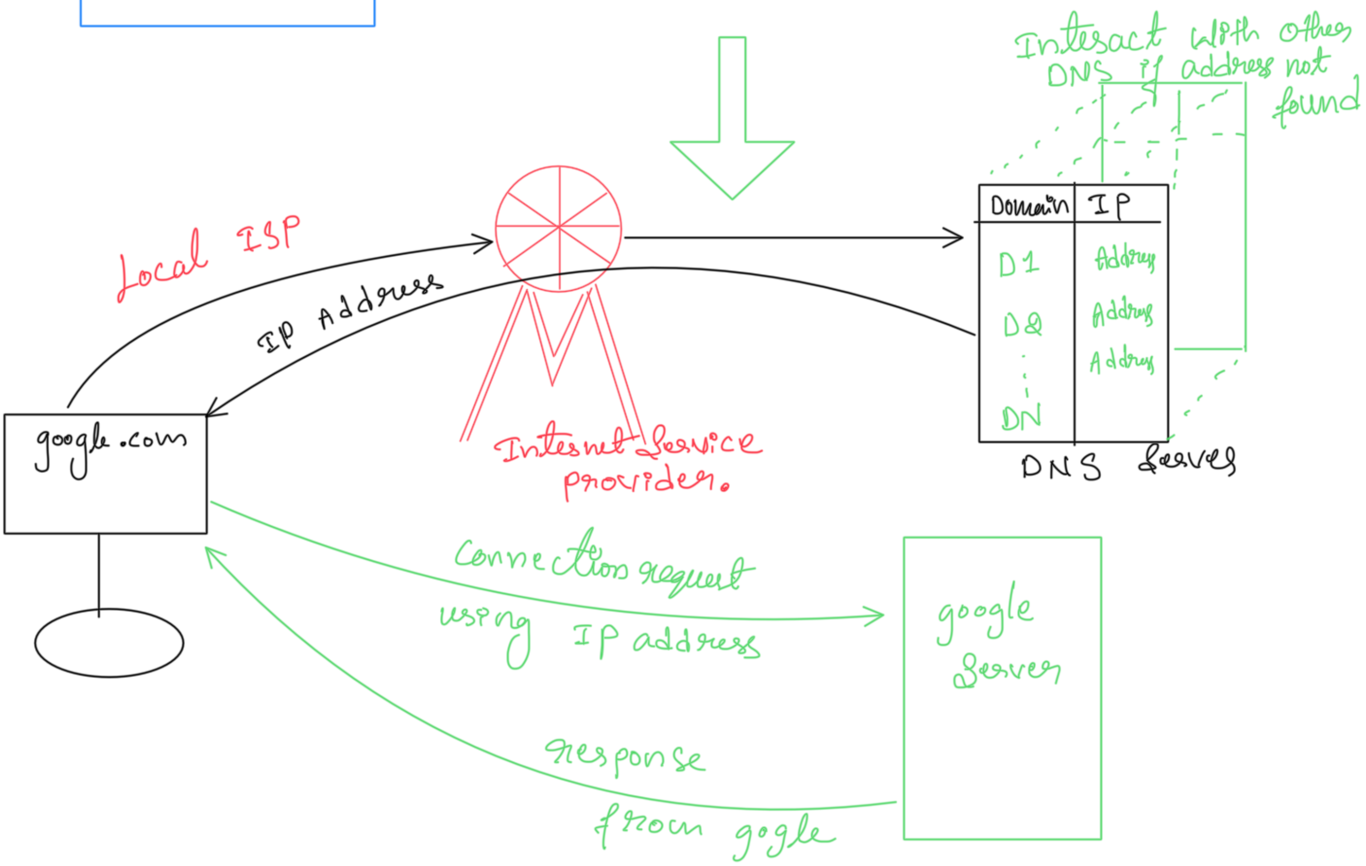
Computer Networks Part - 2

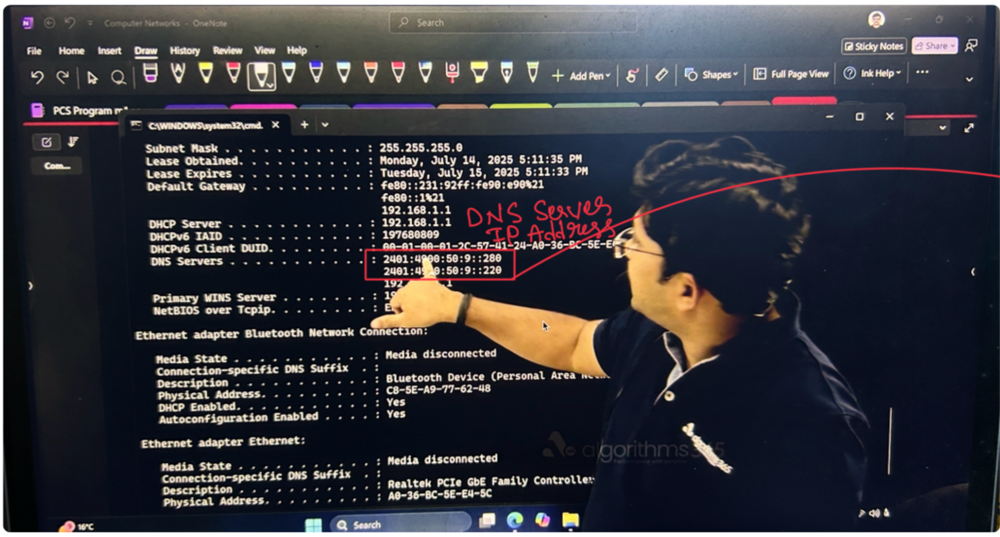
DAY 61
09 | 09 | 2025

Internet

What Happens if We Search Something on the Web Browser





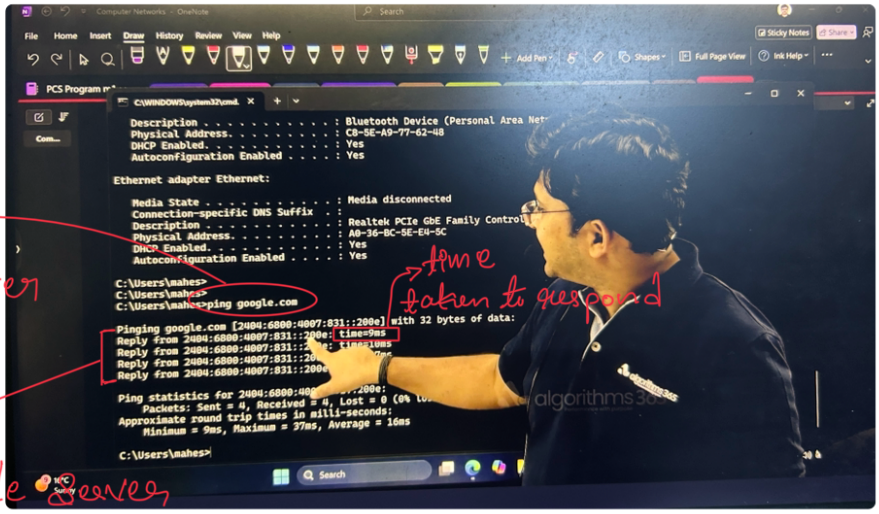


IP Address of DNS Servers

connects to Local ISP

Internet Service Provider.

Asking DNS to Search for google Server IP address in the Internet



DNS returns IPV6 Address of google Server

🔍 **What happens when you type google.com in your browser and press Enter?**

1. Browser Checks Cache (DNS Lookup)

- First, the browser checks if it already knows the IP address of `google.com`.
- It looks into:
  - Browser cache
  - OS cache
  - Local hosts file
  - ISP's DNS cache

2. Establish TCP Connection (Handshake)

- Now your browser knows Google's IP.
- It initiates a **TCP 3-way handshake** with the server:
  - SYN → Client → Server
  - SYN-ACK → Server → Client
  - ACK → Client → Server

👉 This establishes a **reliable communication channel.**

3. TLS/SSL Handshake (If HTTPS)

- Since Google uses **HTTPS**, your browser and server negotiate encryption keys using **TLS**.
- They agree on:
  - Cipher suite
  - Session key
  - Certificates (Google proves it's the real Google with SSL certificate).

4. Browser Sends HTTP Request

- Example request:

```http
GET / HTTP/1.1
Host: www.google.com
```

7. Browser Renders Page

- Browser parses HTML → builds **DOM Tree.**
- Fetches linked CSS/JS files → builds **CSSOM** & executes JS.
- Runs **rendering engine** to paint the UI on screen.

👉 Finally, you see the **Google homepage!**

- This is sent to the server over the secure TCP/TLS connection.

5. Google's Server Processes Request

- Google's load balancers route your request to the nearest and least-loaded data center.
- Web servers + backend services (search engine, ads, personalization) generate a response.

6. Server Sends HTTP Response

- Response includes HTML, CSS, JS, and other resources. Example:

```http
HTTP/1.1 200 OK
Content-Type: text/html
```

- This is sent back in **packets** over TCP/IP.

⚡ Problems Solved by Internet in This Process

- **Packet Switching** → Efficient use of network.
- **DNS** → No need to remember IPs.
- **Encryption (TLS)** → Privacy & security.
- **Load Balancing** → High availability and speed.
- **Caching** → Faster access next time.

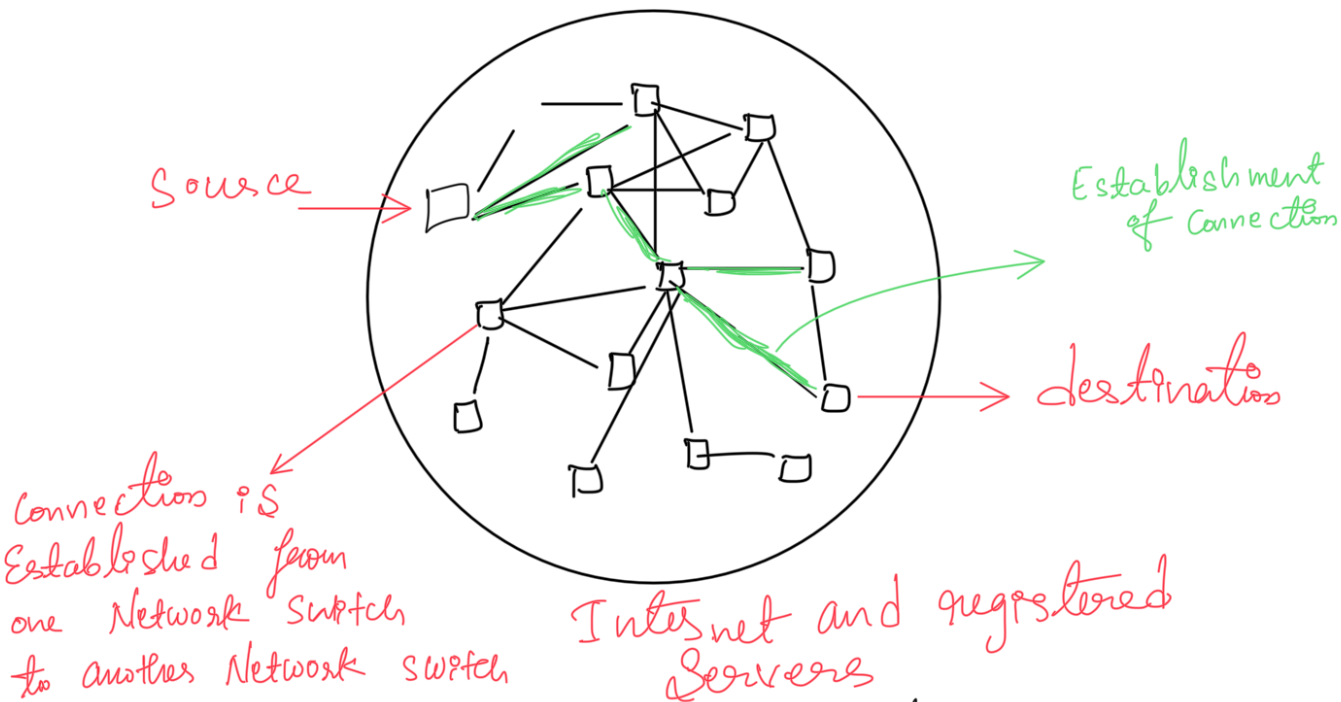
✅ **Summary:**

Typing `google.com` triggers **DNS resolution → TCP/TLS handshake → HTTP request → Server processing → Response → Browser rendering.**

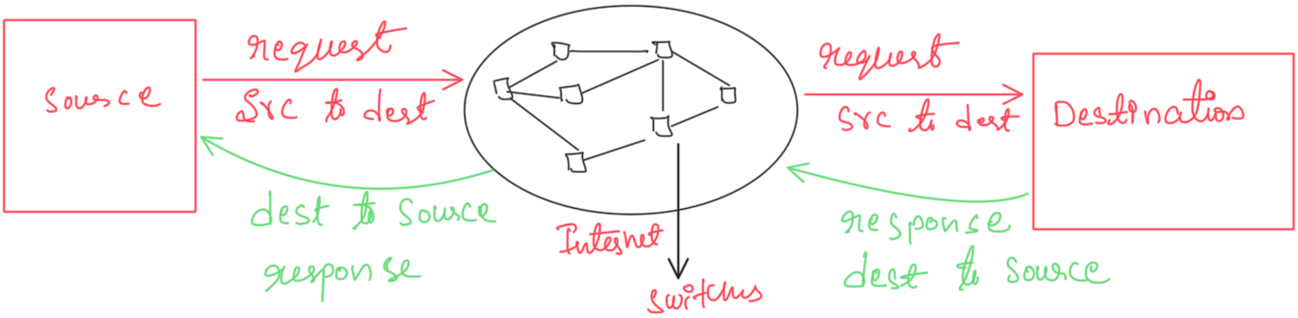
This all happens in a fraction of a second ⚡

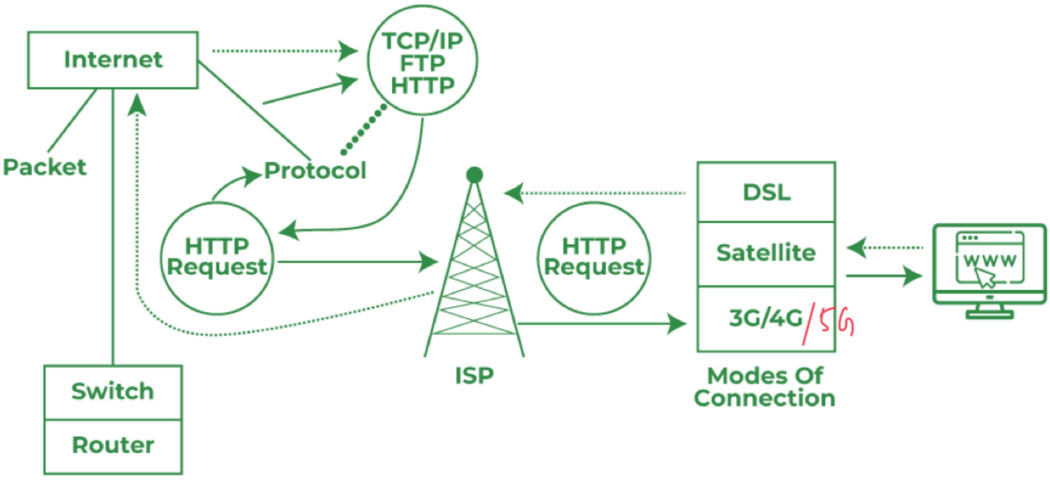
like How a post Man delivers the post to the respective addresses

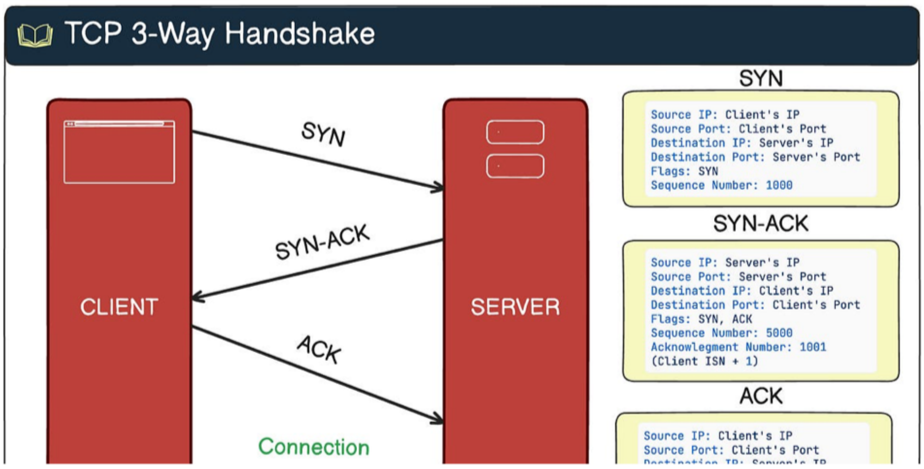
first The letter is collected in local post office after that It will sent to near by post office of that respective address from that It sent to respective district post office then further sent to respective taluk's post office and finally sent to the address.

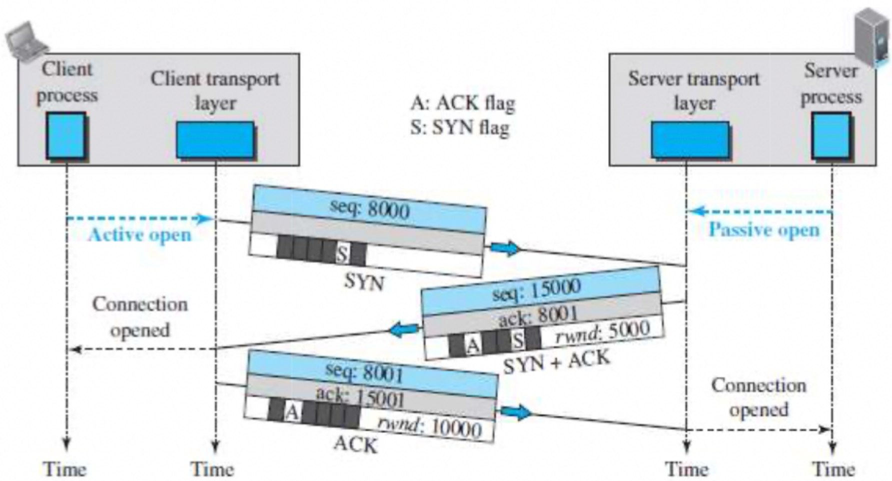


Source will sending the connection establishment request on public IP Address of destination server, In the return the destination server send response for connection establishment if connection is established then the source can request service


Source
request
Src to dest
dest to Source
response
Internet
switches
request
Src to dest
Destination
response
dest to Source


Internet
TCP/IP
FTP
HTTP
Packet
Protocol
HTTP Request
HTTP Request
DSL
Satellite
3G/4G/5G
WWW
ISP
Switch
Router
Modes Of Connection


TCP 3-Way Handshake
CLIENT
SERVER
SYN
SYN-ACK
ACK
Connection
SYN
Source IP: Client's IP
Source Port: Client's Port
Destination IP: Server's IP
Destination Port: Server's Port
Flags: SYN
Sequence Number: 1000
SYN-ACK
Source IP: Server's IP
Source Port: Server's Port
Destination IP: Client's IP
Destination Port: Client's Port
Flags: SYN, ACK
Sequence Number: 5000
Acknowlegment Number: 1001
(Client ISN + 1)
ACK
Source IP: Client's IP
Source Port: Client's Port


Client process
Client transport layer
A: ACK flag
S: SYN flag
Server transport layer
Server process
Active open
seq: 8000
S
SYN
Passive open
Connection opened
seq: 15000
ack: 8001
A
S
rwnd: 5000
SYN + ACK
seq: 8001
ack: 15001
A
rwnd: 10000
ACK
Connection opened
Time
Time
Time
Time

In Hardware Level (using Network driver Software)

Packate

Body	Header

"Hi" message ⟶ ASCII ⟶ Binary ⟶ 100111

Packate

1001101--	101
Body	Header

↓ Sent

Network driver

↓ Sent

Network Hard Ware

Network Card

Cable

1 → High
0 → low

The Binary Info is Sent as Electrical Signal

Through Cable High low Valtage is passed to the network Switch, that Network switch will transfer it further.

In Case of optical fiber Cable light Source is Sent (light ON light off) 1 or 0

NOTE to Send different size of Messages through the Internet, there are different Size of packates

The different Size of packates is called protocols

Example
→ TCP / IP HTTP / HTTPS
→ UDP

📡 1. What are Protocols?

A **protocol** is simply a **set of rules and conventions** that devices follow to communicate over a network.

👉 Just like people need a **common language** (English, Hindi, Kannada, etc.) to talk, computers need **protocols** to understand each other's data exchange.

🔷 2. Why Do We Need Protocols?

Without protocols, every device/vendor would have its own communication style → total chaos 😵.

Protocols bring **standardization, reliability, and efficiency**.

✅ Functions of Protocols

Protocols help with:

1. **Addressing & Routing** → Finding where to send data.
   - Example: **IP (Internet Protocol)** assigns unique addresses.
2. **Reliable Transmission** → Making sure data isn't lost or corrupted.
   - Example: **TCP (Transmission Control Protocol)** handles retransmission & ordering.
3. **Error Detection & Correction** → Ensures accuracy.
   - Example: **Checksums, CRC in Ethernet, TCP.**
4. **Data Formatting & Compression** → Standard way to represent data.
   - Example: **HTTP, SMTP, MP3, JPEG.**
5. **Security & Encryption** → Keeps communication private.
   - Example: **TLS/SSL, HTTPS.**
6. **Flow Control** → Prevents overwhelming slower devices.
   - Example: TCP's windowing mechanism.

Now we can transfer GB's of data

📖 Examples of Protocols

Layer (OSI Model)	Protocols	Purpose
Application	HTTP, HTTPS, FTP, SMTP, DNS	Browsing, file transfer, email, name resolution
Transport	TCP, UDP	Reliable/unreliable data delivery
Network	IP, ICMP, ARP	Addressing & routing packets
Data Link	Ethernet, PPP, Wi-Fi	Local communication
Physical	RS232, DSL, Fiber Optics	Electrical/optical signals

🎯 Real-Life Analogy

Imagine an **international conference** 🌍:

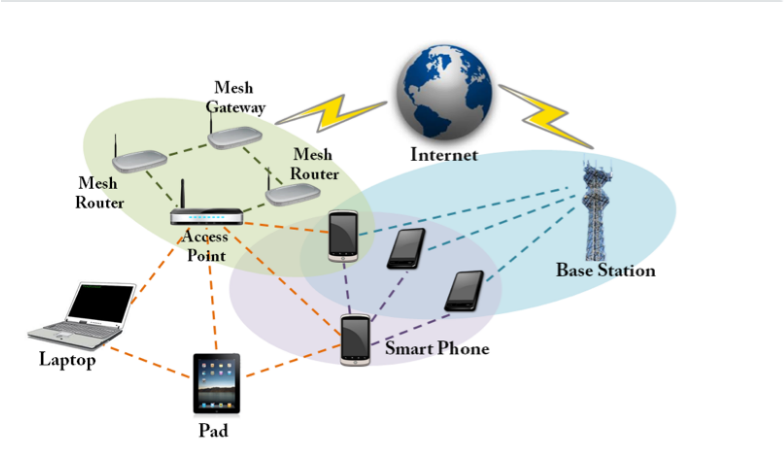
- People from different countries meet.
- If everyone speaks their own language → chaos.
- They agree on **English** (a protocol) → smooth communication.

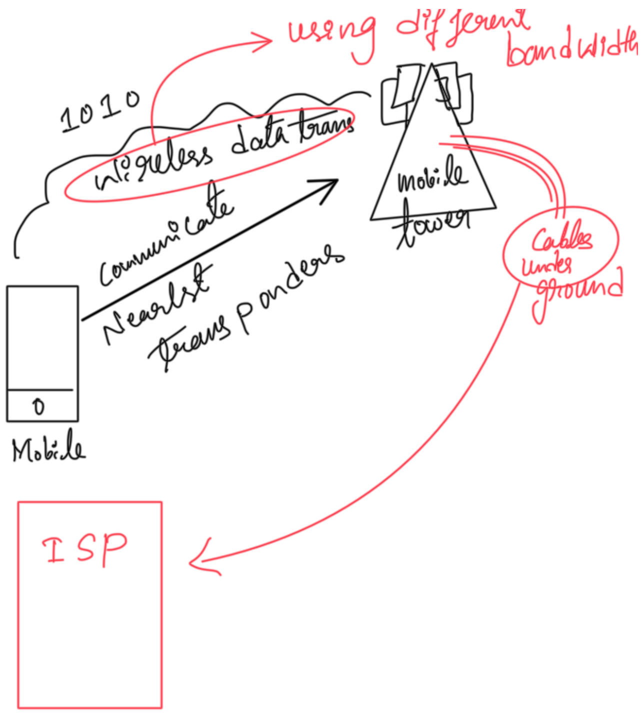
Similarly, **protocols standardize digital communication.**

✅ Summary

- **Protocols = rules for communication** in computer networks.
- Needed for: addressing, reliability, security, error handling, and global standardization.
- Examples: **HTTP, TCP/IP, DNS, FTP, SMTP, TLS, Wi-Fi.**

Wireless Communication





Wi-Fi Connection

📡 Wi-Fi vs 5G

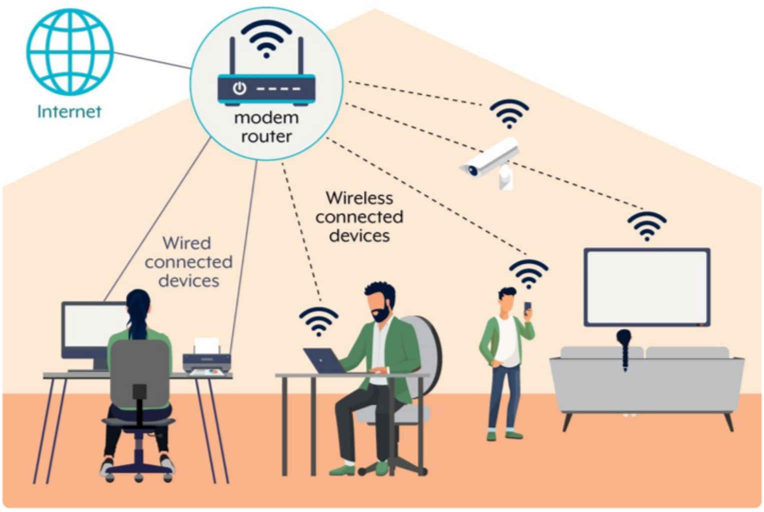
Feature	Wi-Fi	5G
Definition	Local wireless networking technology	5th Generation cellular mobile network
Ownership	Managed by individuals/organizations (home, office, café)	Managed by telecom operators (Airtel, Jio, Verizon)
Coverage Area	Small area (10–50 meters, e.g., home/office)	Wide area (city, country, global)
Technology Base	Uses IEEE 802.11 standards	Uses 3GPP standards for mobile communication
Frequency Bands	2.4 GHz, 5 GHz, now 6 GHz (Wi-Fi 6E)	Licensed spectrum (sub-6 GHz, mmWave up to 100 GHz)
Mobility	Limited mobility (signal drops when leaving Wi-Fi router range)	High mobility (works while traveling in a car/train/plane)
Speed	Wi-Fi 6 → up to ~10 Gbps	5G → up to ~10 Gbps (theoretical)
Latency	~5–20 ms	As low as 1 ms (ultra-low latency)
Cost	Free after setup (you pay only for broadband)	Ongoing data charges from telecom provider
Best Use Cases	Homes, offices, enterprises, indoor high-speed internet	Outdoor internet, mobility (autonomous cars, IoT, smart cities)

🎯 Key Takeaways

- **Wi-Fi** = Best for **local indoor connectivity** (cheap, private).
- **5G** = Best for **outdoor, mobile, large-scale connectivity** (fast, low-latency).
- In reality → They **complement** each other. For example, your phone might use **5G outside** and **Wi-Fi indoors**.

🔍 Real-Life Analogy

- **Wi-Fi** = Your **home water purifier** → gives you water locally at home.
- **5G** = The **city water supply system** → gives you water anywhere in the city.



TCP / IP protocol

📦 1. TCP (Transmission Control Protocol)

👉 **Connection-oriented, reliable, slower**

✅ **Features**

- Ensures data is delivered correctly & in order (retransmission on loss).
- Uses **3-way handshake** before sending data.
- Provides **error checking, flow control, congestion control.**

📍 Where TCP is Used?

Anywhere reliability is more important than speed:

- 🌐 **Web browsing (HTTP/HTTPS)** → pages must load completely.
- 📧 **Email (SMTP, IMAP, POP3)** → emails must be delivered fully.
- 📂 **File transfer (FTP, SFTP)** → no corruption allowed.
- 🖥️ **Remote login (SSH, Telnet)** → commands must execute correctly.
- 📊 **Databases** → query results must be consistent.

⚡ 2. UDP (User Datagram Protocol)

👉 **Connectionless, unreliable, faster**

✅ **Features**

- No handshake, no retransmission → **faster, lightweight.**
- Packets may arrive out of order, or get lost.
- Preferred for **real-time applications** where speed > reliability.

📍 Where UDP is Used?

Anywhere speed and low-latency matter more than 100% accuracy:

- 🎮 **Online gaming** → a missed packet is okay, but lag is not.
- 📺 **Live video/audio streaming (YouTube Live, Zoom, VoIP)** → smooth playback > perfect reliability.
- 🌐 **DNS (Domain Name System)** → fast queries, small packets.
- 🎥 **Video conferencing (Google Meet, MS Teams, WhatsApp calls).**
- 📡 **IoT devices** → small, fast status updates.

🔍 Quick Comparison Table

Feature	TCP	UDP
Type	Connection-oriented	Connectionless
Reliability	Reliable (ack, retransmission)	Unreliable
Speed	Slower	Faster
Overhead	High (headers, checks)	Low
Use Cases	Web, email, file transfer, SSH	Gaming, streaming, VoIP, DN

🎯 Simple Analogy

- **TCP** = Sending a registered parcel 📦 with tracking → safe but slow.
- **UDP** = Sending a postcard 📝 → fast, cheap, but no guarantee it arrives.